

Measuring Algorithm Efficiency

Is insertion sort a good algorithm? This question is hard to answer unless we're sure about what we mean by "good." Insertion sort works—it sorts lists—so it's good in the sense that it accomplishes its purpose. Another point in its favor is that it's easy to understand and explain with reference to physical tasks that many people are familiar with. Yet another feather in its cap is that it doesn't take too many lines of code to express. So far, insertion sort seems like a good algorithm.

However, insertion sort has one crucial failing: it takes a long time to perform. The code in [Listing 4-2](#) almost certainly ran in less than one second on your computer, so the "long time" that insertion sort takes is not the long time that it takes for a tiny seed to become a mighty redwood or even the long time that it takes to wait in line at the DMV. It's more like a long time in comparison to how long it takes a gnat to flap its wings once.

To fret about a gnat's wing flap as a "long time" may seem a little extreme. But there are several good reasons to push algorithms as close as possible to zero-second running times.

Why Aim for Efficiency?

The first reason to relentlessly pursue algorithm efficiency is that it can increase our raw capabilities. If your inefficient algorithm takes one minute to sort an eight-item list, that may not seem like a problem. But consider that such an inefficient algorithm might take an hour to sort a thousand-item list, and a week to sort a million-item list. It may take a year or a century to sort a billion-item list, or it may not be able to sort it at all. If we make the algorithm better able to sort an eight-item list (something that seems trivial since it saves us only a minute), it may make the difference between being able to sort a billion-item list in an hour rather than a century, which can open up many possibilities. Advanced machine-learning methods like k-means clustering and k-NN supervised learning rely on ordering long lists, and improving the performance of a fundamental algorithm like sorting can enable us to perform these methods on big datasets that would otherwise be beyond our grasp.